PayGuard — Simulation d'un système de paiement mobile et détection de fraude

Rapport de projet — Programmation Python et introduction à la science des données

Salif Soumah — Master 1 Intelligence Artificielle et Sciences des Données

Année universitaire 2025-2026

PayGuard — Rapport de projet

Module : Programmation Python et introduction à la science des données **Formation :** Master 1 Intelligence Artificielle et Sciences des Données (IASD) **Auteur :** Salif Soumah (projet réalisé individuellement ; le sujet prévoit un binôme) **Dépôt :** https://github.com/Salif50/payguard **Année universitaire :** 2025-2026

> Tous les chiffres de ce rapport proviennent d'exécutions réelles du code. Les sorties brutes sont dans `rapport/resultats/`, les figures dans `rapport/figures/`, et l'annexe A donne les commandes qui les reproduisent à l'identique (graine fixée).

Résumé

PayGuard est une bibliothèque Python installable qui simule un opérateur de paiement mobile guinéen et détecte les opérations frauduleuses à l'aide de règles métier, sans apprentissage automatique. Le projet couvre les six phases du sujet :

1. un générateur de transactions synthétiques écrit avec la seule bibliothèque standard (123 142 transactions, 5 000 comptes, six mois, 1,49 % de fraudes réparties en six scénarios) ;
2. un modèle orienté objet (comptes abstraits et polymorphes, plateforme aux transferts atomiques) par lequel passent toutes les opérations générées ;
3. les mécanismes avancés du langage (décorateurs, gestionnaires de contexte, générateurs, itérateurs, logging) ;
4. une persistance SQLite, une analyse exploratoire de six questions et deux calculs équivalents des profils de comptes (Python pur et pandas) ;
5. un moteur de règles extensible (patron Stratégie), configurable en JSON, évalué par un coût métier ;
6. une interface en ligne de commande, 163 tests automatisés (couverture de 96 %) et un notebook de démonstration.

Le réglage retenu détecte **95,3 %** des fraudes pour un coût de **27,3 millions de GNF**, contre 949 millions sans détection et 60,5 millions en signalant tout. Sur un second jeu jamais utilisé pour le réglage, il détecte 95,9 % des fraudes pour 25,6 millions de GNF.

Sommaire

1. Introduction
2. Exigences et traçabilité
3. Architecture et conception
4. Phase 1 — Génération des données
5. Phase 2 — Modèle orienté objet
6. Phase 3 — Python avancé
7. Phase 4 — Persistance et analyse des données
8. Phase 5 — Moteur de détection par règles
9. Phase 6 — Qualité logicielle et livraison
10. Extensions facultatives
11. Limites et perspectives
12. Conclusion
13. Sources et outils

Annexes : A. Reproduire les résultats — B. Configuration retenue — C. Arborescence — D. Exemple de sortie de `detecter`

---

1. Introduction

1.1 Contexte

En Guinée, comme dans le reste de l'Afrique de l'Ouest, le paiement mobile sert aux dépôts et retraits d'espèces chez des agents, aux transferts entre particuliers et aux paiements chez les commerçants. Ces services attirent des fraudes typiques : prise de contrôle d'un compte (vol de téléphone, échange de carte SIM, identifiants hameçonnés), comptes « mules » ouverts pour recevoir puis vider des fonds détournés, rafales de transferts avant que la victime ne réagisse.

Un opérateur fictif veut une bibliothèque Python interne pour :

- simuler le fonctionnement de ses comptes et de ses transactions ;
- conserver et analyser l'historique ;
- signaler les opérations suspectes à l'aide de règles métier explicables.

1.2 Objectifs et contraintes

Le projet évalue la maîtrise de Python, des bases à la programmation orientée objet. Les contraintes principales du sujet sont :

- bibliothèques autorisées : bibliothèque standard, pandas, Matplotlib, pytest, pytest-cov ; les phases 1 à 3 n'utilisent que la bibliothèque standard ; **aucun apprentissage automatique** ;
- résultats reproductibles (graine explicite), code conforme à PEP 8, annoté et documenté ;
- architecture de paquet imposée (`src/payguard/…`), installable avec `pip install -e .` ;
- au moins 25 tests et 70 % de couverture sur les paquets `comptes` et `detection` ;
- une commande `detecter` capable de traiter, devant le jury, un jeu inédit de 20 000 transactions sans étiquette.

1.3 Démarche

Le travail a suivi l'ordre des six phases, chacune réutilisant la précédente. Chaque phase a été conçue, implémentée, testée, puis vérifiée par l'exécution réelle avant de passer à la suivante. Les choix non évidents sont justifiés dans les sections correspondantes.

---

2. Exigences et traçabilité

Le tableau suivant relie chaque exigence du sujet au code qui la réalise et à la façon dont elle a été vérifiée.

Exigence du sujet	Réalisation	Vérification
Générateur en fonctions, bibliothèque standard	`generation/generateur.py`, `scenarios.py`, `parametres.py`	`test_generation.py` (20 tests)
≥ 100 000 transactions, plusieurs milliers de comptes, plusieurs mois	123 142 transactions, 5 000 comptes, 6 mois	test sur les paramètres par défaut
4 types d'opérations, montants en GNF selon le type	lois log-normales par type	test des médianes par type
1 à 2 % de fraudes, ≥ 4 scénarios	1,49 %, 6 scénarios	test du taux et de la présence des 6 scénarios
Générateur paramétrable, entrées validées	`valider_parametres` → `ParametreInvalide`	10 cas invalides testés
Format de l'annexe A	`schema.COLONNES`,	test de l'en-tête et des

Exigence du sujet	Réalisation	Vérification
	`ecrire_csv`	colonnes
`Compte` abstrait et 3 classes filles polymorphes	`comptes/base.py`, `personnel.py`, `marchand.py`, `agent.py`	tests de polymorphisme (frais, plafonds)
`Client` validé par propriétés (téléphone guinéen, nom)	`comptes/client.py`	11 numéros testés
`Transaction` immuable, `TypeTransaction`	dataclass figée, `StrEnum`	test `FrozenInstanceError`
Exceptions métier	`exceptions.py`	test de la hiérarchie
`Plateforme`, transfert atomique	`comptes/plateforme.py` + `TransactionAtomique`	test « débit réussi puis crédit refusé »
Méthodes spéciales, collection parcourable, filtrable, triable	`Compte`, `RegistreComptes`, `Plateforme`	tests `len`, `in`, `sorted`, `filtrer`
Générateur refondu pour alimenter la plateforme	`generation/simulation.py`	215 opérations rejetées par les règles métier
3 décorateurs dont un paramétré	`outils/decorateurs.py`	tests de métadonnées, de nouvelles tentatives
Gestionnaire de contexte en classe et avec `contextlib`	`outils/contextes.py`	tests de succès et d'échec pour les deux versions
Lecture par blocs, fenêtre glissante	`outils/flux.py`	tests des bords de la fenêtre
`logging` à la place de `print`	`journal.py` et `getLogger` partout	aucun `print` hors de la CLI
`pyproject.toml`, `pip install -e .`	`pyproject.toml`	installation dans un environnement vierge
`Depot` SQLite (lots, requêtes paramétrées)	`donnees/depot.py`	test aller-retour, test d'injection SQL
≥ 5 questions d'analyse avec pandas et Matplotlib	6 questions, `donnees/analyse.py`	6 figures produites
`ProfilsComptes` en Python pur et pandas, comparés et chronométrés	`donnees/profils.py`	test d'égalité, benchmark
`Regle` abstraite, ≥ 5 règles paramétrables	7 règles dans `detection/regles.py`	tests par règle
`MoteurDeRegles` (Stratégie), `Alerte`	`detection/moteur.py`, `alerte.py`	test d'une règle ajoutée sans modifier le moteur
Configuration JSON validée	`detection/configuration.`	17 configurations invalides

Exigence du sujet	Réalisation	Vérification
	py	testées
Évaluation écrite à la main, coût métier	`detection/evaluation.py`	test sur un cas calculé à la main
CLI : `generer`, `simuler`, `analyser`, `detecter`, `evaluer`	`cli.py`	`test_cli.py` (exécution réelle)
≥ 25 tests, ≥ 70 % de couverture	163 tests ; `comptes` 97,0 %, `detection` 97,7 %	`pytest --cov`
README, notebook sans logique métier	`README.md`, `notebooks/demo.ipynb`	notebook exécuté sans erreur

---

3. Architecture et conception

3.1 Organisation du paquet

Le paquet suit l'arborescence imposée (annexe C). Aucun module imposé n'a été retiré ; cinq modules ont été **ajoutés** parce qu'ils séparent mieux les responsabilités :

Module ajouté	Raison
`schema.py`	Le format de l'annexe A est partagé par la phase 1 (sans classes), le modèle objet et la lecture en flux. Le placer à part évite que le générateur dépende des classes de la phase 2.
`journal.py`	Configuration unique de `logging`. Les modules de la bibliothèque n'y posent aucun gestionnaire : c'est l'application (CLI, notebook) qui décide où écrire.
`generation/parametres.py`	Constantes et tirages aléatoires partagés par le générateur et les scénarios ; évite un import circulaire entre ces deux modules.
`generation/simulation.py`	Refonte de la phase 2 : les opérations générées sont exécutées par la plateforme.
`comptes/registre.py`	La collection de comptes (parcours, filtre, tri) est une responsabilité distincte de la plateforme.

Le graphe des dépendances entre paquets est **acyclique** :

- `generation` (phase 1) → `schema` ;
- `comptes` → `outils.contextes` (qui ne connaît pas les comptes, voir 3.3) ;

- `donnees` → `comptes` ;
- `detection` → `donnees`, `comptes` ;
- `cli` → tous les paquets.

pandas et Matplotlib ne sont importés que là où ils servent (`donnees.analyse`, et à la demande dans `donnees.profils` et `donnees.depot`) : `import payguard` reste léger.

3.2 Diagramme de classes

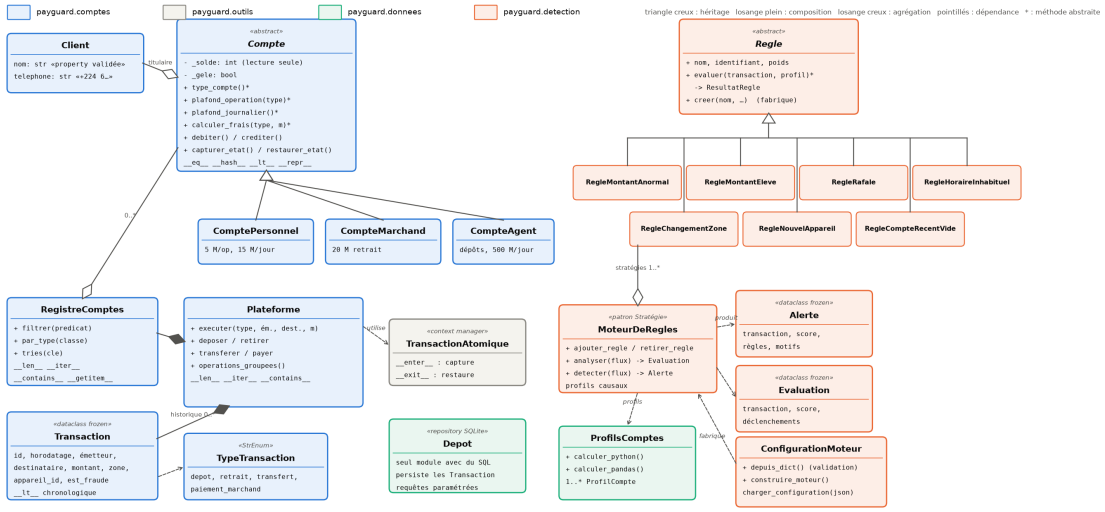


Diagramme de classes simplifié de PayGuard (généré par `rapport/diagramme_classes.py`).

Lecture du diagramme :

- `Compte` est **abstraite** ; `ComptePersonnel`, `CompteMarchand` et `CompteAgent` en héritent et fournissent plafonds, frais et types d'opérations autorisés.
- Un compte **agrège** son titulaire `Client`. `RegistreComptes` agrège les comptes, et `Plateforme` **compose** un registre et l'historique de ses `Transaction`.
- `Plateforme` **utilise** `TransactionAtomique` pour rendre chaque opération atomique.
- `MoteurDeRegles` agrège une ou plusieurs stratégies `Regle` (sept règles concrètes) ; il s'appuie sur les profils de comptes et produit des `Evaluation` et des `Alerte`. `ConfigurationMoteur` fabrique le moteur à partir du JSON.
- `Depot` est le seul module qui contient du SQL et persiste les transactions, les comptes et les alertes.

3.3 Décisions de conception justifiées

1. **Méthode patron dans `Compte`.** `verifier_debit` exécute toujours les contrôles dans le même ordre : gel du compte, type d'opération autorisé, plafond par opération, plafond journalier, solde suffisant **frais compris**. Les classes filles ne redéfinissent que ce qui varie (plafonds, frais, types), au moyen de méthodes

abstraites. On évite de dupliquer la séquence de contrôle, et donc d'oublier un contrôle dans une sous-classe.

2. **Encapsulation du solde.** Le solde n'a qu'un accesseur en lecture (`@property`) ; seules `debiter` et `crediter` le modifient, après validation. Un montant invalide (nul, négatif, non entier, booléen) lève `ParametreInvalide`.
3. **Égalité, hachage et ordre par identifiant.** Deux objets qui décrivent le même compte sont égaux, et `__hash__` est cohérent avec `__eq__` (un compte peut être clé de dictionnaire). L'ordre naturel par identifiant rend `sorted(comptes)` déterministe ; pour trier par solde, on passe une clé explicite (`registre.tries(lambda c: c.solde)`), ce qui évite un ordre « naturel » ambigu.
4. **Atomicité par mémento.** Plutôt que d'écrire, pour chaque opération, une compensation (« recréditer si le crédit échoue »), on capture l'état de chaque participant (solde, cumul journalier, gel) avant l'opération et on le restaure si une exception survient. La plateforme n'ajoute la transaction à l'historique et les frais à ses revenus qu'**après** les deux jambes. `outils.contextes` ne dépend que d'un protocole (`capturer_etat` / `restaurer_etat`), pas des comptes : aucune dépendance circulaire.
5. **Convention émetteur / destinataire.** L'émetteur est le compte **débité**. Pour un dépôt, c'est l'agent (son fonds de roulement électronique passe au client) ; pour un retrait, c'est le client. Cette convention donne un sens uniforme aux profils, qui décrivent les habitudes d'émission.
6. **Transaction immuable.** Une transaction enregistrée est un fait : la dataclass figée interdit toute modification et rend l'objet hachable. Son ordre naturel est chronologique (`__lt__`). `est_fraude` vaut `None` quand l'étiquette est inconnue, ce qui est le cas du jeu de la soutenance.
7. **Exceptions métier hiérarchisées.** Toutes héritent de `PayGuardError`, ce qui permet à la CLI de les intercepter en un seul `except` et de renvoyer un message clair avec le code de sortie 1. Les erreurs de saisie héritent aussi de `ValueError` ou `LookupError`, pour rester compatibles avec le code Python idiomatique.

```
PayGuardError
├── SoldeInsuffisant          ├── ParametreInvalide (ValueError)
├── PlafondDepasse            ├── DonneesInvalides (ValueError)
├── CompteGele                ├── ConfigurationInvalide (ValueError)
├── CompteIntrouvable (LookupError)
├── CompteDejaExistant        └── DepotError
└── OperationNonAutorisee
```

3.4 Plafonds et frais retenus

Ces valeurs sont fictives mais plausibles pour la Guinée. Elles sont portées par des méthodes polymorphes ; en changer revient à modifier une seule classe.

	Particulier	Marchand	Agent
Peut émettre	retrait, transfert, paiement	retrait, transfert	dépôt, transfert
Peut recevoir	dépôt, transfert	paiement, dépôt, transfert	retrait, transfert
Plafond par opération	5 M GNF	20 M (retrait), 10 M (transfert)	10 M (dépôt)
Plafond journalier	15 M GNF	50 M GNF	500 M GNF
Plafond de solde	50 M GNF	aucun	aucun
Frais	transfert 1 %, retrait 1,5 %, paiement gratuit	retrait 1 %, transfert 0,5 %	aucun

---

4. Phase 1 — Génération des données

4.1 Méthode

Le générateur est écrit **en fonctions**, avec `random`, `csv`, `datetime` et `math`. Toute l'aléa provient d'une instance `random.Random(graine)` passée explicitement de fonction en fonction ; l'état global du module `random` n'est jamais utilisé. Deux exécutions avec les mêmes paramètres produisent donc un fichier identique, octet pour octet, ce que vérifie un test automatisé.

La génération se déroule en cinq étapes.

1. **Population de comptes** (`generer_comptes`). 3 % d'agents, 10 % de marchands, 87 % de particuliers ; chaque région a au moins un agent et un marchand. Chaque compte reçoit :
    - une zone principale, tirée selon le poids démographique des huit régions (Conakry, Kindia, Boké, Labé, Mamou, Kankan, Faranah, Nzérékoré), et une zone secondaire ;
    - un ou deux appareils ;
    - une heure d'activité habituelle (loi normale centrée entre 9 h et 19 h) ;
    - une échelle de montants et un niveau d'activité suivant des lois log-normales. Le niveau d'activité a un sigma de 1, ce qui rend l'activité très inégale entre comptes, comme dans la réalité ;
    - un réseau de 3 à 8 contacts, surtout dans sa zone, vers lesquels il transfère.

    Un quart des particuliers ouvre son compte **pendant** la période ; quelques-uns servent de « mules » aux fraudeurs.
2. **Opérations légitimes** (`generer_operations_normales`). Le nombre d'opérations d'un compte est proportionnel à son activité et à sa durée d'existence sur la période. Les montants suivent une loi log-normale propre au type (médianes de base : dépôt

250 000, retrait 200 000, transfert 120 000, paiement 45 000 GNF), multipliée par l'échelle du compte et arrondie à 500 GNF.

3. **Cohérence des soldes** (`equilibrer_soldes`). Les opérations sont rejouées dans l'ordre chronologique avec un solde estimé de façon prudente. Une opération légitime non couverte est précédée d'un dépôt de réapprovisionnement chez un agent de la zone. 2 % d'entre elles sont volontairement laissées **sans provision**, pour que la plateforme de la phase 2 les rejette. Une fraude non couverte est plafonnée au solde disponible : un fraudeur ne peut vider que ce qui existe.
4. **Fraudes** (`injecter_fraudes`). Les scénarios sont tirés au hasard jusqu'à atteindre le taux visé (section 4.2).
5. **Identifiants et écriture** (`attribuer_identifiants`, `ecrire_csv`). Les opérations sont triées par horodatage, numérotées `TX00000001`, `TX00000002`… puis écrites au format de l'annexe A. Un fichier annexe `scenarios.csv` (hors format d'échange) associe chaque fraude à son scénario, pour mesurer la détection par scénario.

4.2 Scénarios de fraude

Six scénarios sont implémentés, soit plus que les quatre exigés. Chacun est une fonction pure enregistrée dans un dictionnaire `SCENARIOS` : en ajouter un ne demande pas de modifier le générateur.

Scénario	Ce qui est injecté	Fraudes acceptées (graine 42)
`montant_eleve`	6 à 20 fois le montant habituel du compte ; nouvel appareil dans 40 % des cas	153
`rafale`	3 à 6 transferts espacés de 20 à 150 secondes vers des mules ; nouvel appareil dans 30 % des cas	618
`horaire_inhabituel`	1 ou 2 opérations entre 0 h et 5 h, 1,5 à 6 fois le montant habituel	250
`changement_zone`	retrait dans une région lointaine 15 min à 2 h 30 après une opération légitime	170
`compte_neuf_vide`	compte ouvert depuis moins de 5 jours qui reçoit des fonds puis en sort 90 à 97 % dans l'heure	452

Scénario	Ce qui est injecté	Fraudes acceptées (graine 42)
`nouvel_appareil`	appareil jamais vu, 2 à 10 fois le montant habituel, autre zone dans 30 % des cas	186

Éviter des fraudes trop faciles. Chaque scénario ne porte qu'**une partie** des signaux possibles, et de façon probabiliste ; inversement, le générateur introduit du **bruit légitime** qui ressemble à de la fraude : 4 % de clients couche-tard, 6 % de changements de téléphone en cours de période, 12 % de comptes à deux appareils, des opérations hors de la zone habituelle (8 %) et des gros montants occasionnels. Aucune règle isolée ne peut donc séparer parfaitement fraudes et opérations légitimes (section 8.4).

4.3 Validation des paramètres

`valider_parametres` lève `ParametreInvalide` avec un message explicite si : le nombre de comptes n'est pas un entier entre 50 et 500 000 ; le nombre d'opérations n'est pas entre 100 et 10 000 000 ; les dates ne sont pas des dates ou couvrent moins de 30 jours ; le taux de fraude n'est pas dans [0 ; 0,2] (ou vaut NaN) ; la graine n'est pas un entier. Les booléens, qui sont des entiers en Python, sont refusés explicitement.

4.4 Jeu de données produit

Paramètre ou résultat (graine 42)	Valeur
Comptes	5 000 (150 agents, 500 marchands, 4 350 particuliers)
Période	1er janvier – 30 juin 2026
Transactions brutes (`payguard generer`)	123 142, dont 1 830 fraudes (1,49 %)
Transactions acceptées par la plateforme (`payguard simuler`)	122 927, dont 1 829 fraudes (1,49 %)
Opérations rejetées par les règles métier	215 (`SoldeInsuffisant`)
Appareils distincts	5 882

Répartition par type des transactions acceptées :

Type	Transactions	Fraudes	Taux de fraude	Volume (GNF)
Dépôt	33 481	0	0,00 %	21,7 milliards
Retrait	25 849	534	2,07 %	12,2 milliards
Transfert	37 732	1 155	3,06 %	10,2 milliards
Paiement marchand	25 865	140	0,54 %	1,9 milliard

Les fraudes portent toutes sur des sorties d'argent : aucun scénario n'utilise un dépôt, car un dépôt d'espèces chez un agent ne fait pas sortir d'argent du compte de la victime.

5. Phase 2 — Modèle orienté objet

5.1 Classes principales

- **`Client`** : `nom` et `telephone` sont des propriétés validées à la construction **et** à chaque modification ; un client ne peut jamais être dans un état invalide. Le téléphone accepte les formats usuels (`620 12 34 56`, `+224 620…`, `00224…`) et est normalisé en `+2246XXXXXXXX` : 9 chiffres commençant par 6, comme les numéros mobiles guinéens.
- **`Compte`** (abstraite) et ses trois classes filles, décrites en 3.3 et 3.4.
- **`Transaction`** : dataclass figée, avec `slots`, validée dans `__post_init__`. Elle se construit depuis une ligne CSV (`depuis_ligne`, tolérante aux variations de casse, aux montants `1500.0`, aux fuseaux horaires) et s'y reconvertit (`vers_ligne`).
- **`TypeTransaction`** : énumération `StrEnum`. Chaque membre *est* une chaîne, ce qui simplifie l'écriture CSV et SQLite.
- **`RegistreComptes`** : collection indexée par identifiant (accès en temps constant). Elle implémente `__len__`, `__iter__`, `__contains__` (par identifiant ou par objet) et `__getitem__`, plus `filtrer`, `par_type` (polymorphisme : sous-classes comprises) et `tries`.
- **`Plateforme`** : compose le registre et l'historique, et expose `deposer`, `retirer`, `transferer`, `payer`, qui passent tous par `executer`.

5.2 Déroulement atomique d'une opération

`Plateforme.executer` suit toujours la même séquence :

1. résoudre l'émetteur et le destinataire (`CompteIntrouvable` sinon) ;
2. vérifier qu'ils sont différents et que le destinataire peut recevoir ce type d'opération ;
3. ouvrir une `TransactionAtomique` sur les deux comptes ;
4. débiter l'émetteur (montant + frais), créditer le destinataire, construire la `Transaction` ;
5. si tout a réussi : ajouter les frais aux revenus et la transaction à l'historique ; sinon, le gestionnaire de contexte restaure les deux comptes, le motif du rejet est compté, puis l'exception se propage.

Le test central gèle le destinataire : le débit de l'émetteur réussit, puis le crédit échoue avec `CompteGele`. Après l'exception, les soldes **et le cumul journalier** de l'émetteur sont identiques à l'état initial, l'historique est vide et la plateforme n'a encaissé aucun frais. Un second test provoque l'échec par le plafond de solde du destinataire. Un troisième vérifie la **conservation de la monnaie** : la masse monétaire plus les revenus de la plateforme restent constants.

`operations_groupees()` étend l'atomicité à plusieurs opérations (par exemple un paiement fractionné vers deux commerçants) : si la seconde échoue, la première est annulée elle aussi.

5.3 Générateur refondu : les données passent par les règles métier

`generation/simulation.py` instancie un objet `Compte` par compte généré, puis fait exécuter chaque opération par `Plateforme.executer`. Seules les opérations acceptées constituent le jeu de données final. Avec la graine 42, **215 opérations** sont rejetées (`SoldeInsuffisant`), toutes légitimes sauf une ; les frais encaissés par la plateforme s'élèvent à 250 384 384 GNF. La simulation complète, génération comprise, prend environ 2 secondes.

---

6. Phase 3 — Python avancé

6.1 Décorateurs

Décorateur	Rôle	Utilisation dans le projet
`@chronometre`	Mesure la durée avec `time.perf_counter`, la journalise et l'expose dans `derniere_duree`	`simuler`, calcul des profils, insertion SQLite
`@journaliser`	Trace l'appel, le résultat et les exceptions au niveau DEBUG, puis relance l'exception	`Plateforme.executer`
`@reessayer(n, delai, exceptions, facteur)`	Décorateur **paramétré** : réessaie uniquement sur les exceptions déclarées transitoires, avec un délai qui peut croître de façon exponentielle	ouverture de la base SQLite (`OperationalError`, base verrouillée)

`@reessayer` comporte trois niveaux de fonctions : la fabrique reçoit les paramètres et les valide, le décorateur reçoit la fonction, l'enveloppe exécute les tentatives. Les trois décorateurs utilisent `functools.wraps`, qui conserve le nom, la docstring et la signature ; un test le vérifie avec `inspect.signature`. `@journaliser` ne formate ses arguments que si le niveau DEBUG est actif, pour ne rien coûter en temps normal (la plateforme exécute plus de 120 000 opérations).

6.2 Gestionnaires de contexte

Le même mécanisme de transaction atomique est écrit deux fois, comme le demande le sujet :

- `TransactionAtomique` : `__enter__` capture l'état des participants, `__exit__` le restaure dans l'ordre inverse si une exception survient, puis renvoie `False` pour **ne jamais avaler l'exception** ;
- `transaction_atomique` : générateur décoré par `contextlib.contextmanager`, dont le `yield` est entouré d'un `try/except` qui restaure puis relance.

Les deux versions sont testées avec les mêmes scénarios de succès et d'échec (tests paramétrés).

6.3 Générateurs et itérateur

- `lire_transactions` est un **générateur** : il lit le CSV ligne à ligne et produit des `Transaction` validées, sans jamais charger le fichier. Une ligne invalide est ignorée, comptée et journalisée (seules les dix premières en avertissement, pour ne pas inonder la console), ou lève `DonneesInvalides` en mode strict. Il tolère un BOM UTF-8 et les lignes vides, et signale le numéro de ligne physique de chaque erreur.
- `lire_par_blocs` regroupe ce flux en listes de taille fixe (`itertools.islice`) : la mémoire utilisée est bornée par la taille d'un bloc.
- `FenetreGlissanteParCompte` est un **itérateur** écrit à la main (`__iter__`, `__next__`) qui associe à chaque transaction les dernières transactions du même compte. Au début, les fenêtres sont partielles (sauf avec `completes_seulement=True`) ; en fin de flux, l'itération s'arrête sans produire de fenêtre vide ; une durée maximale peut borner la fenêtre dans le temps. Ces trois comportements aux bords sont testés.

6.4 Journalisation et packaging

- `logging` remplace `print` dans toute la bibliothèque. La CLI configure un fichier de journal (`data/payguard.log`, niveau INFO ou DEBUG avec `-v`) et une console limitée aux avertissements ; seuls les **résultats** destinés à l'utilisateur sont affichés avec `print`, dans `cli.py`.
- `pyproject.toml` décrit le paquet (dépendances, extras `dev` et `notebook`, licence MIT) et déclare la commande `payguard`. L'installation avec `pip install -e ".[dev]"` a été vérifiée dans un environnement vierge : 33 modules s'importent sans erreur.

---

7. Phase 4 — Persistance et analyse des données

7.1 Le dépôt SQLite

`Depot` (patron *Repository*) est le **seul** module du projet qui contient du SQL :

- schéma avec contraintes (`CHECK` sur les types et sur `montant > 0`), clés primaires et index (`id_emetteur, horodatage`) ;
- insertion **par lots** de 5 000 lignes avec `executemany`, dans une transaction SQLite, à partir de n'importe quel itérable (un générateur convient) ;

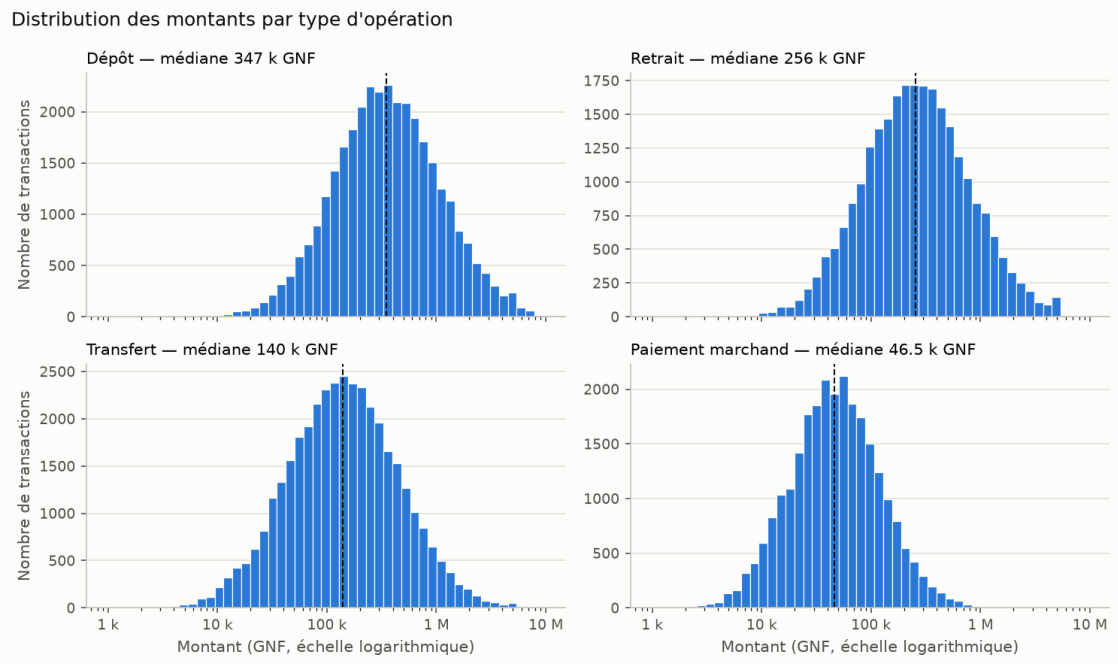
- requêtes **paramétrées** exclusivement : un test vérifie qu'une valeur comme `P1' OR '1'='1` est traitée comme une simple donnée ;
- horodatages stockés en ISO 8601 : l'ordre du texte coïncide avec l'ordre chronologique, si bien que `ORDER BY` et les bornes de dates fonctionnent directement ;
- utilisable comme gestionnaire de contexte (validation ou annulation, puis fermeture).

La base produite par `payguard simuler` contient 122 927 transactions et 5 000 comptes, et peut aussi recevoir les alertes (`detecter --base`).

7.2 Analyse exploratoire

Six questions sont traitées, pour cinq exigées. Chaque question a une fonction de calcul (testable sans affichage) et une fonction de tracé.

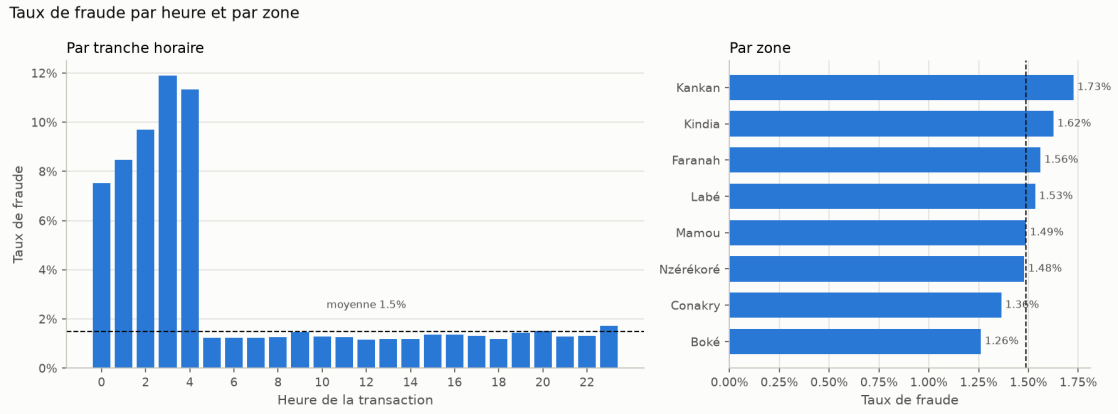
Q1 — Distribution des montants par type (échelle logarithmique).



Distribution des montants par type d'opération.

Les distributions sont log-normales et bien séparées : médianes de 347 000 GNF pour les dépôts, 256 000 pour les retraits, 140 000 pour les transferts et 46 500 pour les paiements. Un seuil de montant unique n'a donc pas le même sens pour tous les types.

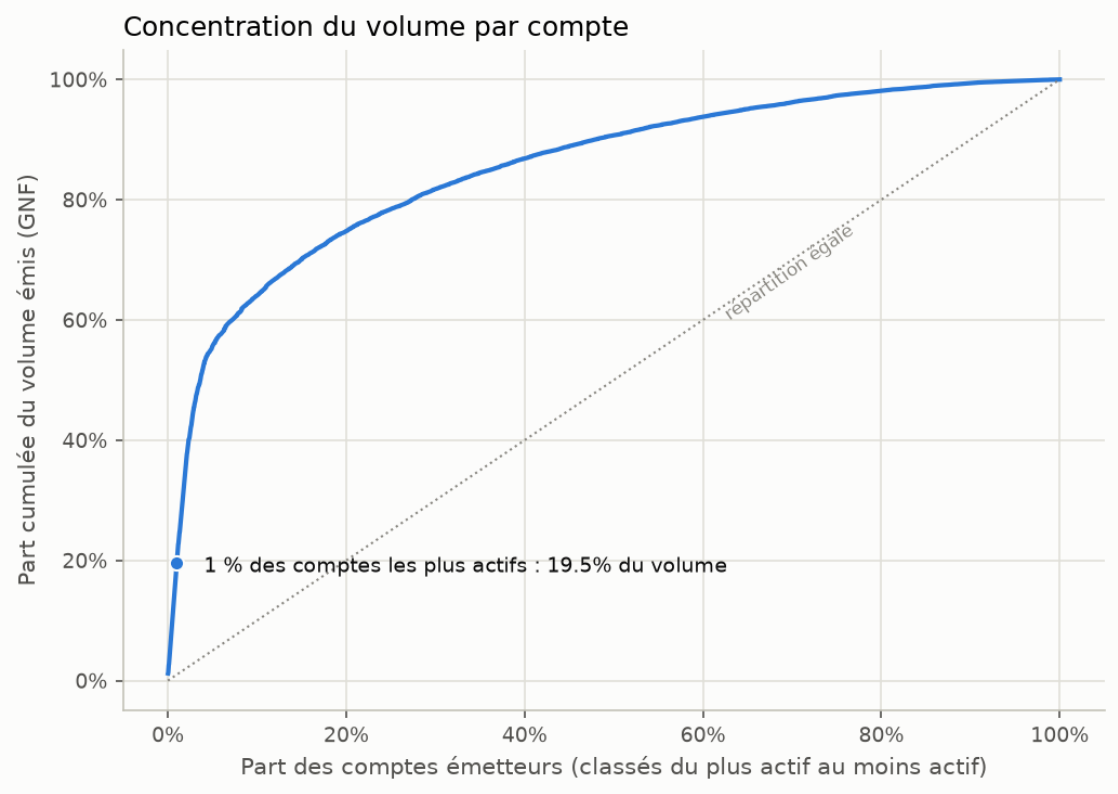
Q2 — Taux de fraude par tranche horaire et par zone.



Taux de fraude par heure et par zone.

Entre 0 h et 5 h, le taux de fraude atteint 7,5 à 11,9 %, contre 1,2 à 1,7 % le reste de la journée. Selon la zone, l'écart est faible (de 1,26 % à Boké à 1,73 % à Kankan) : la zone seule n'est pas un signal, c'est son **changement** qui compte. Cela justifie les règles `horaire_inhabituel` et `changement_zone`.

Q3 — Part du volume portée par le centile de comptes le plus actif.

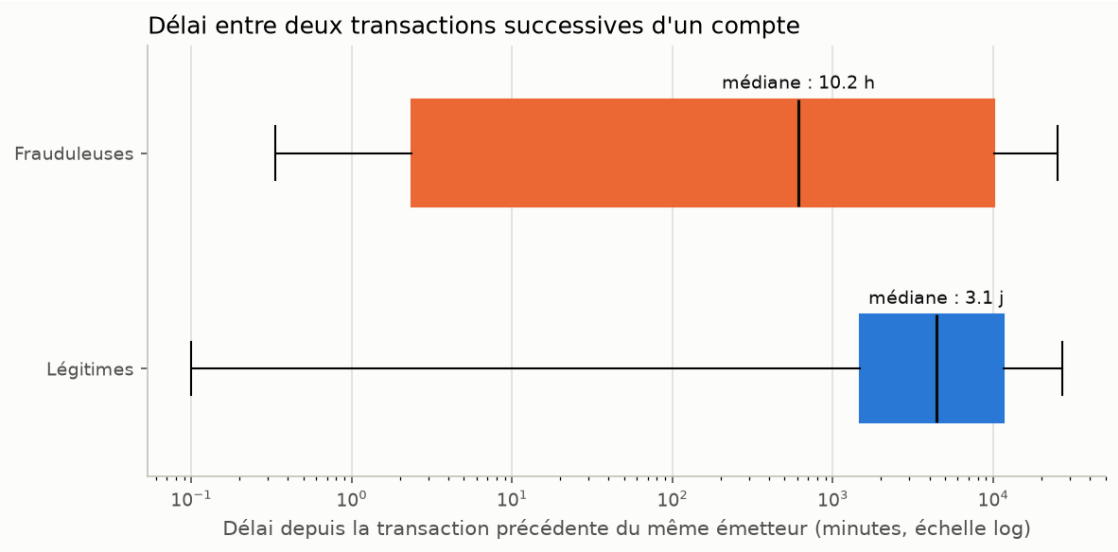


Concentration du volume émis par compte.

Les 49 comptes les plus actifs (1 % des 4 882 émetteurs) portent **19,5 %** du volume émis et 12,2 % des opérations ; 43 d'entre eux sont des agents. Ces comptes ont des habitudes

très différentes des particuliers : les règles doivent raisonner relativement au profil de chaque compte.

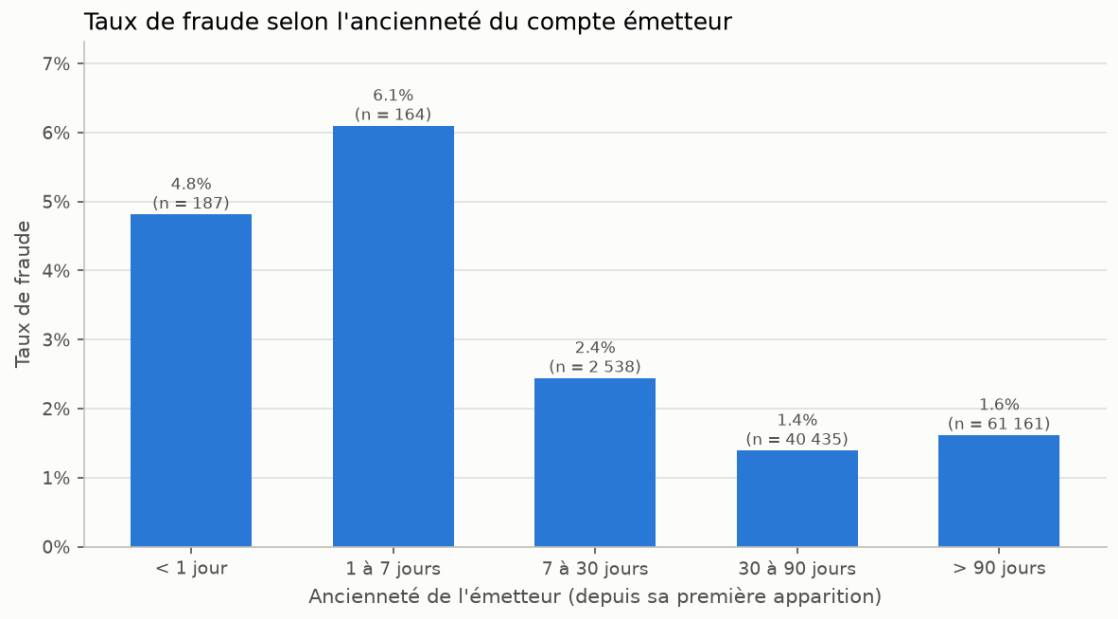
Q4 — Délai médian entre deux transactions successives d'un même émetteur.



Délai depuis la transaction précédente du même émetteur.

Le délai médian est de **10,2 heures** pour les fraudes contre **3,1 jours** pour les opérations légitimes. Les fraudes suivent de près l'opération précédente du compte (rafales, voyages impossibles, comptes vidés), ce qui justifie les règles `rafale` et `changement_zone`. Les dépôts, émis par les agents, sont exclus de cette comparaison, car leur activité de guichet écraserait le résultat.

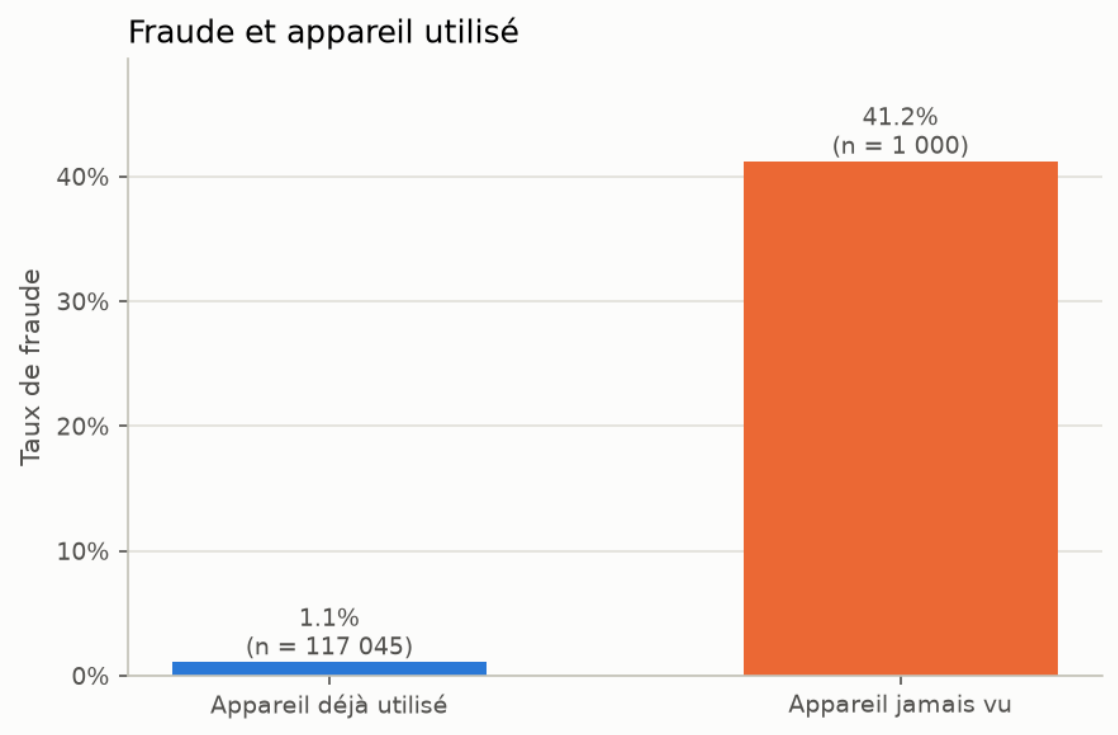
Q5 — Lien entre ancienneté du compte et fraude.



Taux de fraude selon l'ancienneté de l'émetteur.

Le fichier ne contient pas de date d'ouverture : l'ancienneté est mesurée depuis la première apparition du compte dans les données. Les 30 premiers jours sont écartés, car tous les comptes y paraissent récents (biais de censure à gauche). Résultat : 4,8 % de fraude pour les comptes apparus depuis moins d'un jour, 6,1 % entre 1 et 7 jours, contre 1,4 à 1,6 % au-delà de 30 jours. Cela justifie la règle `compte_recent_vide`.

Q6 (question libre) — La fraude est-elle plus fréquente depuis un appareil jamais vu ?



Taux de fraude selon que l'appareil est connu ou non du compte.

Pourquoi cette question ? Le changement d'appareil est le signe le plus direct d'une prise de contrôle de compte (vol de téléphone, échange de carte SIM, identifiants hameçonnés). Il fallait mesurer à la fois son pouvoir de détection et le nombre de clients honnêtes qu'il dérangerait. Le calcul est causal : pour chaque transaction, on regarde si l'appareil avait déjà servi à ce compte **auparavant**.

Résultat : **41,2 %** de fraude depuis un appareil jamais vu, contre 1,1 % depuis un appareil connu. Ces cas ne représentent que 1 000 transactions, mais 25 % des fraudes. La règle `nouvel_appareil` est donc l'une des plus précises (section 8.4).

7.3 Profils de comptes : Python pur contre pandas

`ProfilsComptes` calcule pour chaque compte : le nombre d'opérations émises, le montant total, moyen et maximal, les heures d'activité, les zones et appareils utilisés (`Counter`), la dernière zone et la dernière opération, le total reçu et la première apparition.

- `calculer_python` : un tri puis une seule passe, avec des dictionnaires et des `Counter` ;
- `calculer_pandas` : groupby et agg sur les émetteurs et les destinataires, puis conversion en objets `ProfilCompte`.

Les deux résultats sont **égaux** (== sur les 4 999 profils, champ par champ), ce que vérifient un test et la commande `payguard analyser --benchmark`.

Mesure (122 927 transactions, meilleur de 5 essais, Python 3.12, Apple Silicon)	Temps
Python pur (tri compris)	0,114 s
pandas, agrégation vectorisée seule	0,061 s
pandas, agrégation et conversion en objets `ProfilCompte`	0,095 s

Commentaire. L'agrégation pandas est environ 1,9 fois plus rapide que la version Python. Une fois la conversion en objets comptée, l'écart tombe à 1,2 fois : à ce volume, le coût dominant n'est plus le calcul mais la création de 5 000 objets et de leurs `Counter`. La version Python reste très rapide (une passe O(n) après un tri en O(n log n)). pandas prendrait nettement l'avantage sur des volumes bien plus grands, ou si l'on restait en DataFrame sans revenir à des objets. Les deux versions reçoivent des données déjà chargées : on mesure le calcul, pas la lecture du fichier. Ces temps dépendent de la machine et doivent être remesurés sur la machine de soutenance.

---

8. Phase 5 — Moteur de détection par règles

8.1 Conception

- **`Regle` (abstraite)** impose `evaluer(transaction, profil)`, qui renvoie un `ResultatRegle` : nom de la règle, déclenchée ou non, poids, motif lisible. Chaque règle valide ses seuils dans son constructeur (`ParametreInvalide` sinon) et peut être restreinte à certains types d'opérations.
- **Patron Stratégie.** `MoteurDeRegles` ne connaît que l'interface `Regle`. Ajouter ou retirer une règle passe par `ajouter_regle` / `retirer_regle` ou par le fichier JSON, **sans modifier le moteur**. Les sous-classes s'enregistrent automatiquement dans un registre (`__init_subclass__`), et la fabrique `Regle.creer(nom, …)` vérifie les noms de paramètres avec `inspect.signature`. Un test écrit une règle « liste noire » après coup et la branche sans toucher au moteur.
- **Identifiants.** Une même stratégie peut être instanciée plusieurs fois sous des identifiants distincts. C'est ainsi qu'on obtient deux paliers de montant tout en gardant un score purement additif.
- **Score et alerte.** Le score est la somme des poids des règles déclenchées. Une transaction est signalée lorsque le score atteint le seuil d'alerte. `Alerte` (dataclass figée) associe la transaction, son score, les règles déclenchées et leurs motifs ; elle est triable par score. `Evaluation` conserve le score de **toutes** les transactions, ce qui permet d'évaluer plusieurs seuils sans relancer les règles.

8.2 Profils causaux : aucune fuite de données

C'est le point le plus important de la conception. `MoteurDeRegles.analyser` parcourt le flux dans l'ordre chronologique ; chaque transaction est évaluée avec le profil **antérieur** de son émetteur, puis intégrée à ce profil. Par conséquent :

- aucune information future n'influence une décision, et le moteur ne lit **jamais** `est_fraude`. Un test inverse toutes les étiquettes, puis les supprime, et vérifie que les alertes sont strictement identiques ;
- le même code traite un fichier inconnu sans étiquette, comme le jeu de la soutenance ;
- le moteur peut fonctionner au fil de l'eau sur un flux non chargé en mémoire (`detecter --flux`), et produit alors exactement les mêmes alertes qu'en mode trié (test) ;
- un historique de référence peut initialiser les profils (`--historique`) ; il est copié, jamais modifié.

8.3 Les règles

Sept règles sont implémentées, dont deux instances de `montant_eleve` ; elles couvrent les six scénarios.

Règle (identifiant)	Se déclenche si…	Paramètres retenus	Poids
`montant_anormal`	montant ≥ facteur × moyenne du compte **et** au-dessus de son maximum passé	facteur 4, ≥ 3 opérations passées, montant ≥ 100 000 GNF	1
`rafale`	au moins *n* opérations émises dans la fenêtre	2 opérations en 5 minutes	2
`horaire_inhabituel`	heure dans la plage surveillée, plage ≤ 10 % de l'historique du compte	0 h – 5 h	1
`changement_zone`	autre zone moins de 3 h après la dernière opération (« voyage impossible »), ou zone jamais utilisée par un	délai 180 min	1

Règle (identifiant)	Se déclenche si…	Paramètres retenus	Poids
	compte ayant ≥ 5 opérations		
`nouvel_appareil`	appareil jamais utilisé par un compte ayant ≥ 2 opérations	—	2
`compte_recent_vide`	compte apparu depuis ≤ 7 jours qui émet ≥ 80 % de ses fonds reçus et non dépensés	montant ≥ 50 000 GNF	1
`montant_significatif`	sortie d'argent ≥ 50 000 GNF (dépôts exclus)	stratégie `montant_eleve`	1
`montant_eleve`	sortie d'argent ≥ 150 000 GNF (dépôts exclus)	stratégie `montant_eleve`	1

Seuil d'alerte : **2**. Chaque alerte porte des motifs lisibles, par exemple : « appareil APP-1A2B3C4D jamais utilisé par le compte (1 appareil(s) connu(s), 14 opérations) » ou « opération à Kankan 42 min après une opération à Conakry ».

8.4 Pouvoir de chaque règle prise seule

Chaque règle a été évaluée **seule**, chaque déclenchement valant alerte, sur le jeu de la graine 42 (`rapport/resultats/regles_individuelles.txt`) :

Règle seule	Déclenchements	Dont fraudes	Précision	Détection	Coût (GNF)
`montant_anormal`	3 598	230	6,4 %	12,6 %	704 604 500
`rafale`	915	468	51,1 %	25,6 %	839 041 000
`horaire_inhabituel`	1 999	217	10,9 %	11,9 %	857 398 500
`changement_zone`	3 235	201	6,2 %	11,0 %	841 549 000
`nouvel_appareil`	772	367	47,5 %	20,1 %	718 528 500
`compte_recent_vi`	1 013	41	4,0 %	2,2 %	929 952 500

Règle seule	Déclenchements	Dont fraudes	Précision	Détection	Coût (GNF)
`montant_de_significatif`	67 031	1 773	2,6 %	96,9 %	34 212 500
`montant_eleve`	38 509	1 514	3,9 %	82,8 %	47 075 000

Trois enseignements guident le réglage :

- `rafale` et `nouvel_appareil` sont de loin les règles les plus **précises** (environ une alerte sur deux est une fraude) : elles méritent un poids suffisant pour alerter seules ;
- les règles comportementales sont précises mais **partielles** : chacune ne voit qu'une partie des fraudes, à cause du bruit légitime et des signaux partiels injectés (section 4.2) ;
- les paliers de montant détectent presque tout, mais au prix de nombreuses fausses alertes. Leur coût est pourtant bas, ce qu'explique la section suivante.

8.5 Évaluation et coût métier

Le module `evaluation` est écrit à la main, sans bibliothèque d'apprentissage automatique. Il calcule la matrice de confusion (fraudes détectées, fraudes manquées, fausses alertes, vrais négatifs), le taux de détection, la précision, le taux de fausses alertes, la charge d'alertes, l'exactitude, le F1 et le **coût métier** :

> coût = somme des montants des fraudes manquées + 500 GNF × nombre d'alertes injustifiées.

Pourquoi l'exactitude n'est pas une mesure recevable. Avec 1,49 % de fraudes, un « détecteur » qui ne signale **rien** obtient 98,51 % d'exactitude, tout en laissant passer 100 % des fraudes, soit 949 millions de GNF de pertes. L'exactitude est dominée par la classe majoritaire et ne pèse pas les erreurs selon leur gravité. Dans le tableau de la section 8.6, le réglage *prudent* a une meilleure exactitude (98,35 %) que le réglage retenu (68,09 %), alors qu'il coûte 21 fois plus cher. On évalue donc séparément les détections, les manques et les fausses alertes, et l'on décide sur le coût.

Ce que révèle le coût. Une fraude manquée coûte en moyenne environ 520 000 GNF, soit autant qu'environ **1 000 fausses alertes**. Le critère pousse donc à signaler largement les sorties d'argent importantes : signaler *toutes* les transactions coûte 60,5 millions de GNF, soit 16 fois moins que ne rien faire.

8.6 Comparaison des réglages et choix

Démarche. (1) Chaque règle a d'abord été mesurée seule (section 8.4). (2) Une recherche sur grille (poids 1 ou 2 pour les six règles comportementales, différents paliers de montant) a été menée sur le jeu de la graine 42. (3) Le meilleur réglage a ensuite été **validé sur la graine 2026**, un jeu jamais utilisé pour le réglage, afin de contrôler le sur-ajustement.

Réglage	Alertes	Détectées	Manquées	Fausses alertes	Détection	Précision	Exactitude	Coût (M GNF)
Référence : aucune alerte	0	0	1 829	0	0 %	—	98,51 %	949,1
Référence : tout signaler	122 927	1 829	0	121 098	100 %	1,49 %	1,49 %	60,5
Retenu, seuil 2	40 879	1 743	86	39 136	**95,30 %**	4,26 %	68,09 %	**27,3**
Retenu, seuil 1	69 839	1 818	11	68 021	99,40 %	2,60 %	44,66 %	34,3
Retenu, seuil 3	6 598	1 153	676	5 445	63,04 %	17,47 %	95,02 %	359,0
Retenu, seuil 4	1 139	685	1 144	454	37,45 %	60,14 %	98,70 %	600,2
Équilibré (paliers 200 k / 1 M)	9 595	1 266	563	8 329	69,22 %	13,19 %	92,77 %	215,4

Réglage	Alertes	Détectées	Manquées	Fausses alertes	Détection	Précision	Exactitude	Coût (M GNF)
Prudent (sans palier de montant)	1 945	872	957	1 073	47,68 %	44,83 %	98,35 %	584,0
Validation, grain e 2026 : retenu	40 290	1 750	75	38 540	95,89 %	4,34 %	68,46 %	25,6

Justification du réglage retenu. Il est celui qui minimise le coût métier. Concrètement :

- toute sortie d'argent d'au moins 150 000 GNF est signalée seule (les deux paliers de montant cumulent 2 points). Les dépôts sont exclus, car un dépôt d'espèces chez un agent ne fait pas perdre d'argent ;
- entre 50 000 et 150 000 GNF, il faut **un indice** comportemental supplémentaire ;
- `rafale` et `nouvel_appareil`, les deux règles les plus précises, déclenchent seules quel que soit le montant.

Il divise le coût par **35** par rapport à l'absence de détection et par **2,2** par rapport à « tout signaler ». Sur le jeu de validation, il obtient des résultats légèrement meilleurs (95,89 % de détection, 25,6 millions de GNF) : il ne s'est pas sur-ajusté au jeu de réglage. Le seuil 2 est un optimum : à 1, les fausses alertes en plus coûtent davantage que les fraudes en moins ; à 3 et au-delà, les fraudes manquées explosent.

Détection par scénario (réglage retenu).

Scénario	Fraudes	Détectées	Taux	Montant manqué (GNF)
`changement_zone`	170	168	98,8 %	50 500
`nouvel_appareil`	186	183	98,4 %	232 500
`horaire_inhabituel`	250	240	96,0 %	778 500
`compte_neuf_vide`	452	428	94,7 %	2 320 500
`montant_eleve`	153	144	94,1 %	476 000
`rafale`	618	580	93,9 %	3 855 500

La rafale est le scénario le moins bien couvert, et c'est inévitable pour une règle causale : au premier transfert d'une rafale, rien ne permet encore de savoir qu'une rafale commence.

Contribution des règles parmi les alertes du réglage retenu.

Règle	Déclenchements sur fraudes	Déclenchements sur opérations légitimes
`montant_significatif`	1 704	38 710
`montant_eleve`	1 514	36 995
`rafale`	468	447
`nouvel_appareil`	367	405
`montant_anormal`	230	2 923
`horaire_inhabituel`	213	735
`changement_zone`	199	2 299
`compte_recent_vide`	41	862

Limite assumée du critère. Le réglage retenu signale environ **un tiers** des transactions. Ce n'est raisonnable que si le « traitement » à 500 GNF d'une alerte est automatique, par exemple une confirmation envoyée au client par code à usage unique. Si les alertes étaient examinées par des analystes en nombre limité, il faudrait minimiser le coût **sous contrainte de capacité**. Le réglage *prudent*, avec 1 945 alertes et 45 % de précision, serait alors plus pertinent ; il reste disponible dans `config/reglages/prudent.json`.

8.7 Configuration JSON

Le moteur est construit depuis un fichier JSON (annexe B) : règles actives, identifiants, poids, paramètres et seuil d'alerte. La validation est stricte et chaque erreur lève `ConfigurationInvalide` avec un message qui désigne l'élément fautif :

- JSON mal formé (ligne et colonne indiquées) ou racine qui n'est pas un objet ;
- clé inconnue (faute de frappe), seuil absent, nul, négatif ou non numérique ;
- liste de règles vide, règle inconnue, doublon d'identifiant, aucune règle active ;
- poids négatif, `active` non booléen, paramètre inconnu ou hors bornes.

Ces 17 cas d'erreur sont testés. Une configuration acceptée est forcément utilisable, car toutes les règles sont instanciées pendant la validation. Si aucun fichier n'est fourni, la CLI utilise `config/regles.json`, sinon une configuration intégrée identique.

9. Phase 6 — Qualité logicielle et livraison

9.1 Interface en ligne de commande

Commande	Rôle	Exemple
`generer`	CSV brut de la phase 1	`payguard generer --comptes 5000 --graine 42`
`simuler`	Exécution sur la plateforme, CSV des transactions acceptées, base SQLite	`payguard simuler --graine 2026 --sans-base`
`analyser`	Six analyses, figures, comparaison des profils	`payguard analyser --benchmark`
`detecter`	Alertes et règles déclenchées pour chaque transaction signalée, CSV des alertes	`payguard detecter jeu.csv --details`
`evaluer`	Métriques, coût, comparaison de réglages, détection par scénario, évolution mensuelle	`payguard evaluer -c a.json b.json --seuils 1 3`

Chaque commande a une aide (`--help`), des valeurs par défaut documentées et une validation des arguments (dates au format AAAA-MM-JJ, taux dans [0 ; 0,2], fichiers existants…). Codes de sortie : 0 succès, 1 erreur métier (message clair sur la sortie d'erreur), 2 arguments invalides, 130 interruption.

Robustesse de `detecter` pour le jeu inédit. La commande a été testée sur un fichier imitant celui de la soutenance : sans colonne `est_fraude`, dans le désordre, avec un BOM, une ligne vide, une date invalide, un type inconnu et un type écrit en majuscules. Elle trie le flux, ignore et signale les lignes invalides (avec leur numéro de ligne) et produit ses alertes sans échouer. Elle traite les 122 927 transactions du jeu complet en environ 1 seconde.

9.2 Tests et couverture

163 tests pytest, tous réussis (environ 5 à 10 secondes) :

Fichier	Tests	Contenu principal
`test_generation.py`	20	reproductibilité, format, volume, taux, scénarios, paramètres invalides
`test_comptes.py`	39	client, polymorphisme, plafonds, gel, exceptions, transaction, registre
`test_plateforme.py`	14	opérations, frais,

Fichier	Tests	Contenu principal
		atomicité, conservation de la monnaie, simulation
`test_outils.py`	20	décorateurs, contextes, lecture en flux, fenêtre glissante
`test_donnees.py`	11	SQLite, injection SQL, profils Python = pandas, analyses
`test_detection.py`	52	règles, moteur, causalité, alertes, configuration, évaluation, coût
`test_cli.py`	7	les cinq commandes exécutées réellement, codes de sortie

Couverture mesurée avec pytest-cov : **96 %** au total, **97,0 %** sur le paquet `comptes` et **97,7 %** sur le paquet `detection` (70 % exigés). Les cas d'erreur sont testés au même titre que les cas nominaux.

Les tests ont révélé deux défauts réels, corrigés :

- des dates de création générées avec des microsecondes, perdues à l'écriture du CSV (précision à la seconde) : l'aller-retour CSV puis SQLite n'était pas exact ;
- des numéros de ligne erronés dans les messages d'erreur lorsque le CSV contient des lignes vides.

9.3 Style, documentation et démonstration

- Code conforme à PEP 8, avec des lignes de 99 caractères au plus (limite admise par PEP 8, vérifiée avec `pycodestyle`) ; annotations de types sur les interfaces publiques ; docstrings en français.
- Aucune dépendance hors de la liste autorisée : `numpy`, installé avec pandas, n'est jamais importé directement.
- `README.md` : installation en moins de dix minutes, commandes, architecture, résultats, reproductibilité, limites. `CONTRIBUTING.md` et licence MIT pour le dépôt public.
- `notebooks/demo.ipynb` : parcours complet des six phases, de la génération à la détection sur un fichier inconnu. Il n'appelle que des fonctions du paquet et s'exécute de bout en bout sans erreur.

9.4 Reproductibilité

Les commandes de l'annexe A régénèrent les fichiers à l'identique. Empreintes SHA-256 obtenues :

```
a0f65621e3ec9555ddb38f754dc66a37c930ebc352a02e39a71c3511df194a37  data/raw/transactions.csv
d92db4b7ca593adf9aadb8c48a2ed2e55026615f3440c60e5fb86f928090f429  data/processed/transactions.csv
```

---

10. Extensions facultatives

Deux des extensions proposées par le sujet ont été réalisées :

- **Détection au fil de l'eau** : `payguard detecter fichier.csv --flux` lit les transactions en flux et le moteur émet ses alertes au fur et à mesure, sans charger le fichier (qui doit alors être trié par horodatage). Un test vérifie que les alertes sont identiques à celles du mode trié.
- **Évolution du coût au fil des mois** : `payguard evaluer --mensuel`.

Mois (2026)	Alertes	Détection	Précision	Coût (GNF)
Janvier	6 348	94,90 %	2,93 %	4 007 500
Février	5 859	92,89 %	3,57 %	4 071 000
Mars	6 882	95,31 %	3,84 %	4 422 000
Avril	6 976	95,65 %	4,42 %	4 585 500
Mai	7 386	94,99 %	4,36 %	5 106 500
Juin	7 428	96,60 %	6,11 %	5 089 000

La précision double entre janvier et juin : à mesure que l'historique s'allonge, les profils deviennent plus fiables et les règles comportementales plus discriminantes. Le coût mensuel augmente légèrement avec le volume d'activité, plus élevé en fin de période (comptes ouverts en cours de période).

---

11. Limites et perspectives

- **Circularité des données synthétiques.** Les règles ont été conçues en connaissant les scénarios ; les performances sur de vraies fraudes, ou sur le jeu de l'enseignant, seront plus faibles. La validation sur une seconde graine contrôle le sur-ajustement aux tirages, pas aux scénarios eux-mêmes.
- **Seuils absolus en GNF.** Les paliers de 50 000 et 150 000 GNF dépendent de la distribution des montants. Perspective : des seuils relatifs au profil du compte, ou aux quantiles du fichier analysé.

- **Ancienneté estimée** par la première apparition dans les données, faute de date d'ouverture dans le format d'échange : au début d'un fichier, tous les comptes paraissent récents.
- **Démarrage à froid.** Sur un fichier court comme le jeu de la soutenance (20 000 transactions), les comptes ont peu d'historique : les règles comportementales se déclenchent moins et les paliers de montant pèsent plus. L'option `--historique` permet d'initialiser les profils lorsqu'un historique est disponible.
- **Convention émetteur / destinataire.** Un fichier qui suivrait une autre convention pour les dépôts resterait lisible, mais les profils des agents et des clients y seraient mélangés.
- **Coût métier à deux niveaux.** Un modèle plus réaliste distinguerait le coût d'une vérification automatique de celui d'une enquête humaine, et intégrerait la capacité de traitement des analystes.
- **Perspectives** : une interface interactive en console, une API HTTP minimale de détection sur une transaction isolée et un tableau de bord statique, trois extensions du sujet non réalisées.

---

12. Conclusion

PayGuard réalise les six phases du sujet sous la forme d'une bibliothèque Python installable, testée (163 tests, 96 % de couverture) et reproductible. La conception orientée objet repose sur quelques idées simples et justifiées : une classe abstraite qui fixe l'ordre des contrôles, des classes filles qui ne définissent que ce qui varie, une atomicité obtenue en capturant puis restaurant l'état, un moteur qui ne connaît que l'interface de ses règles.

L'analyse exploratoire a guidé le choix des règles (heure, appareil, délai, ancienneté). L'évaluation a montré qu'avec le coût métier imposé, l'exposition financière (le montant) compte autant que les indices comportementaux : le réglage retenu détecte 95 % des fraudes et divise le coût par 35 par rapport à l'absence de détection, au prix d'un volume d'alertes qui suppose un traitement automatique. Enfin, la conception causale du moteur garantit qu'il fonctionne à l'identique sur un fichier inconnu et sans étiquette, comme lors de la soutenance.

---

13. Sources et outils

Conformément au sujet, je déclare les sources et outils utilisés au-delà d'une simple consultation de documentation.

Source ou outil	Usage
Site officiel de Python (docs.python.org)	Documentation de référence : `abc`, `dataclasses`, `enum`, `contextlib`, `functools`,

Source ou outil	Usage
	`csv`, `sqlite3`, `logging`, `argparse`.
Claude (Anthropic), via l'assistant de programmation Claude Code	Assistant de programmation utilisé pour écrire le code du paquet, les tests, la configuration du moteur, le notebook, le README, le diagramme de classes et ce rapport, à partir du sujet et de mes consignes. Les résultats chiffrés proviennent tous d'exécutions réelles du code (sorties dans `rapport/resultats/`).
ChatGPT (OpenAI)	Assistance à la structuration et à la mise en forme du rapport Word à partir du rapport Markdown.
`black`, `pycodestyle`	Outils de développement, non requis à l'exécution : mise en forme du code et vérification de la conformité PEP 8.

J'ai relu le code produit avec l'aide de ces assistants, je l'ai exécuté et testé. Je m'engage à pouvoir en expliquer chaque partie lors de la soutenance.

Annexe A — Reproduire les résultats

```
python3 -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]"

payguard generer --graine 42                    # data/raw/transactions.csv
(123 142 lignes)
payguard simuler --graine 42                    # data/processed/transaction
s.csv (122 927) + data/payguard.db
payguard analyser --benchmark --repetitions 5    # analyses, figures, profils
 Python / pandas
payguard evaluer -c config/regles.json config/reglages/equilibre.json config/
reglages/prudent.json \
    --seuils 1 3 4 --scenarios data/processed/scenarios.csv --mensuel

# Jeu de validation
payguard simuler --graine 2026 -o data/validation/transactions.csv \
    --scenarios data/validation/scenarios.csv --sans-base
payguard evaluer -i data/validation/transactions.csv -c config/regles.json

# Tests, couverture, diagramme
pytest --cov=payguard
python rapport/diagramme_classes.py
```

Annexe B — Configuration retenue (`config/regles.json`, extrait)

```
{
  "seuil_alerte": 2.0,
  "regles": [
    {"nom": "rafale", "poids": 2.0,
     "parametres": {"nb_operations": 2, "fenetre_minutes": 5}},
    {"nom": "nouvel_appareil", "poids": 2.0, "parametres": {"min_operations":
 2}},
    {"nom": "montant_eleve", "identifiant": "montant_significatif", "poids":
1.0,
     "parametres": {"seuil": 50000,
                    "types_operations": ["retrait", "transfert", "paiement_ma
rchand"]}},
    {"nom": "montant_eleve", "poids": 1.0,
     "parametres": {"seuil": 150000,
                    "types_operations": ["retrait", "transfert", "paiement_ma
rchand"]}}
  ]
}
```

Le fichier complet contient aussi `montant_anormal`, `horaire_inhabituel`, `changement_zone` et `compte_recent_vide`, avec les paramètres du tableau de la section 8.3.

Annexe C — Arborescence du projet

```
payguard/
├── pyproject.toml, README.md, CONTRIBUTING.md, LICENSE
├── config/regles.json, config/reglages/{equilibre,prudent}.json
├── notebooks/demo.ipynb
├── rapport/  rapport.md, diagramme_classes.py, figures/, resultats/
├── src/payguard/
│   ├── __init__.py, __main__.py, cli.py, exceptions.py, schema.py, journal.p
y
│   ├── generation/  generateur.py, scenarios.py, parametres.py, simulation.p
y
│   ├── comptes/     base.py, personnel.py, marchand.py, agent.py, client.py,
│   │                transaction.py, plateforme.py, registre.py
│   ├── outils/      decorateurs.py, contextes.py, flux.py
│   ├── donnees/     depot.py, profils.py, analyse.py
│   └── detection/   base.py, regles.py, moteur.py, alerte.py, configuration.
py, evaluation.py
└── tests/  conftest.py + 7 fichiers de tests (163 tests)
```

Annexe D — Exemple de sortie de `payguard detecter --details`

Sortie réelle sur le jeu de la graine 42 (`payguard detecter data/processed/transactions.csv --limite 2 --details`):

```
[ALERTE] TX00000002 | 2026-01-01 00:15:19 | transfert | 153 000 GNF | P001384
 -> P002834 | score 3
    - horaire_inhabituel : opération à 00:15, plage 0 h-5 h représentant 0% d
e l'historique du compte
    - montant_significatif : montant 153 000 GNF >= seuil de 50 000 GNF
    - montant_eleve : montant 153 000 GNF >= seuil de 150 000 GNF
[ALERTE] TX00000003 | 2026-01-01 00:15:52 | paiement_marchand | 57 500 GNF |
P000291 -> M000329 | score 2
    - horaire_inhabituel : opération à 00:15, plage 0 h-5 h représentant 0% d
e l'historique du compte
    - montant_significatif : montant 57 500 GNF >= seuil de 50 000 GNF
... 40877 autres alertes (toutes dans data/processed/alertes.csv)

Transactions analysées : 122927  |  lignes ignorées : 0
Alertes : 40879 (33.25%)  ->  data/processed/alertes.csv
Étiquettes présentes (non utilisées par le moteur) : 1743 fraudes détectées,
86 manquées,
39136 fausses alertes, coût métier 27 281 500 GNF
```

Sur un fichier sans colonne `est_fraude`, la dernière ligne est remplacée par un message indiquant que le coût ne peut pas être calculé.